

# Usando *map*

`map(função, iterável)` aplica a mesma função a cada item e devolve um **iterador** com os resultados. É o jeito direto de converter uma lista de textos em números, formatar valores ou transformar dados em série, sem escrever um `for` com `append`. Nesta aula: `map` com funções prontas e com lambda, converter entradas, combinar vários iteráveis, a avaliação preguiçosa e quando a list comprehension é a melhor escolha.

`map(função, iterável)`

`str.upper · int · len`

`map + lambda`

Vários iteráveis

Preguiçoso

`map × comprehension`

Formação Claude Code 2026 · Baseado em docs.python.org (Built-in Functions: map; Functional Programming HOWTO; operator), consultados em outubro de 2026  
· Exemplos testados com Python 3.14 em `exemplos/` · Saídas reais

## A mesma função *em cada item*

basico.py

```
nomes = ["ana", "bia", "caio"]

maiusculos = map(str.upper, nomes)    # aplica em cada item
print(maiusculos)
print(list(maiusculos))
print(list(map(len, nomes)))
```

SAÍDA

```
<map object at 0x7d6fe1ab2f00>
['ANA', 'BIA', 'CAIO']
[3, 3, 4]
```



- ✓ Passe a função **sem parênteses**: `str.upper`, não `str.upper()`
- ✓ O resultado é um objeto `map`, um iterador; `list()` mostra os itens
- ✓ Funções prontas como `len`, `str`, `int` e métodos como `str.strip` funcionam direto

## De texto para *número*

converter.py

```
entrada = "10 25 7 40"          # como viria de um input()

numeros = list(map(int, entrada.split()))
print(numeros, sum(numeros))

precos = map(float, ["9.90", "15", "0.5"])
print(sum(precos))
```

SAÍDA

```
[10, 25, 7, 40] 82
25.4
```

- ✓ `map(int, texto.split())` é o uso mais comum de `map` no dia a dia
- ✓ Útil para ler vários números de um `input()` numa linha só
- ✓ `sum` consome o iterador direto, sem lista intermediária

Se algum item não for número, `int` lança `ValueError`. Para dados de fora, valide antes (aula 07).

## A transformação *que você quiser*

com\_lambda.py

```
precos = [100.0, 59.9, 12.5]

com_reajuste = map(lambda p: round(p * 1.10, 2), precos)
print(list(com_reajuste))

def formatar(valor):
    return f"R$ {valor:.2f}".replace(".", ",")

print(list(map(formatar, precos)))
```

SAÍDA

```
[110.0, 65.89, 13.75]
['R$ 100,00', 'R$ 59,90', 'R$ 12,50']
```

- ✓ Para uma conta curta, uma `lambda` resolve
- ✓ Para algo com nome e mais de uma etapa, escreva uma função com `def` e passe o nome
- ✓ A mesma `formatar` pode ser reaproveitada em outros lugares e testada sozinha

## Funções com *mais de um argumento*

varios.py

```
from operator import mul

bases = [2, 3, 4]
expoentes = [3, 2, 1]

print(list(map(pow, bases, expoentes)))    # pow(2, 3), ...
# para no menor iterável:
print(list(map(mul, [1, 2, 3], [10, 20, 30, 40])))
```

SAÍDA

```
[8, 9, 4]
[10, 40, 90]
```

- ✓ Com dois iteráveis, a função recebe um item de cada: `pow(2, 3)`, `pow(3, 2)`, `pow(4, 1)`
- ✓ Como o `zip`, para no iterável mais curto: o 40 ficou de fora
- ✓ O módulo `operator` tem as operações como funções: `mul`, `add`, `sub` ...

## Calcula *só quando pedem*

preguica.py

```
def dobrar(n):  
    print(f"  dobrando {n}")  
    return n * 2  
  
m = map(dobrar, [1, 2, 3])  
print("map criado")           # nada foi calculado ainda  
print(next(m))  
print(list(m))               # calcula o resto  
print(list(m))               # e se esgota
```

SAÍDA

```
map criado  
  dobrando 1  
2  
  dobrando 2  
  dobrando 3  
[4, 6]  
[]
```

- ✓ Criar o `map` não chama a função nenhuma vez
- ✓ Cada `next()` calcula um item, como nos geradores das aulas 03 e 04
- ✓ Depois de consumido, o `map` está esgotado: a última lista sai vazia

Precisa usar o resultado mais de uma vez? Guarde em lista:

```
lista = list(map(...)) .
```

# Qual *escolher?*

comprehension.py

```
nums = [1, 2, 3, 4]

print(list(map(str, nums)))          # função pronta: curto
print([str(n) for n in nums])

print(list(map(lambda n: n * n, nums)))    # lambda...
print([n * n for n in nums])              # ...lê melhor assim
```

SAÍDA

```
['1', '2', '3', '4']
['1', '2', '3', '4']
[1, 4, 9, 16]
[1, 4, 9, 16]
```

| SITUAÇÃO                                                             | PREFIRA                                |
|----------------------------------------------------------------------|----------------------------------------|
| Função pronta, sem condição                                          | <code>map(int, itens)</code>           |
| Precisaria de lambda                                                 | <code>[x * 2 for x in itens]</code>    |
| Transformar e filtrar juntos                                         | comprehension com <code>if</code>      |
| Resultado vai direto para <code>sum</code> ,<br><code>max</code> ... | <code>map</code> ou expressão geradora |
| Os dois dão o mesmo resultado. Escolha o que fica mais fácil de ler. |                                        |

## Celsius para *Fahrenheit*

**Enunciado:** converta uma lista de temperaturas em °C para °F com `map` e mostre as duas lado a lado.

Uma resposta possível

```
celsius = [-5, 0, 21.5, 37, 100]

def para_fahrenheit(c):
    return c * 9 / 5 + 32

fahrenheit = map(para_fahrenheit, celsius)
for c, f in zip(celsius, fahrenheit):
    print(f"{c:>6} °C = {f:>6.1f} °F")
```

SAÍDA

```
-5 °C = 23.0 °F
 0 °C = 32.0 °F
21.5 °C = 70.7 °F
37 °C = 98.6 °F
100 °C = 212.0 °F
```

- ✓ `zip` casa cada valor original com o convertido
- ✓ `>6.1f` alinha à direita com uma casa decimal
- ✓ Desafio: faça o caminho de volta, de °F para °C

## Teste o que *ficou*

1. `list(map(int, ["1", "2"]))` vale:

A) `["1", "2"]`

B) `[1, 2]`

C) 3

D) Erro

**B.**

2. `print(map(len, ["ab"]))` mostra:

A) `[2]`

B) 2

C) `<map object at ...>`

D) `["ab"]`

**C.** É um iterador.

3. `list(map(pow, [2, 3], [2]))` vale:

A) `[4, 9]`

B) `[4]`

C) `[4, 3]`

D) Erro

**B.** Para no menor iterável.

4. Como passar a função para o `map`?

A) `map(str.upper(), nomes)`

B) `map(str.upper, nomes)`

C) `map(nomes, str.upper)`

D) `map("upper", nomes)`

**B.**

# Transformar *em série*

`map(função, iterável)` aplica a função em cada item e devolve um iterador preguiçoso, que se esgota. Brilha com funções prontas, como `map(int, texto.split())`; com vários iteráveis, para no menor. Quando precisaria de lambda ou de filtro, a list comprehension costuma ler melhor.

`map(f, it)`

Sem parênteses na função

Vários iteráveis

Preguiçoso

Comprehension

Próxima aula → 09 · Usando Filter

Fontes: [docs.python.org/3/library/functions.html#map](https://docs.python.org/3/library/functions.html#map); [docs.python.org/3/howto/functional.html](https://docs.python.org/3/howto/functional.html); [docs.python.org/3/library/operator.html](https://docs.python.org/3/library/operator.html); [docs.python.org/3/library/functions.html#pow](https://docs.python.org/3/library/functions.html#pow), consultados em outubro de 2026. Exemplos em [exemplos/](#), executados com Python 3.14.4.